RETAILMART V3 · SQL

Review & Interview Prep

— Siraj Sir

📱 WhatsApp Announcement

REVIEW & INTERVIEW PREP

---

23 days of SQL. From SELECT * to recursive CTEs to RFM segmentation. Today is RAPID-FIRE PRACTICE — 9 interview-style problems covering every major topic. Plus the 'Explain Your Query' drill that separates juniors from seniors. Tomorrow we start the capstone.

Today's Focus:

- 9 rapid-fire interview problems (one per major topic)
- Explain-your-query drill — narrate any query line by line
- 6 mixed-difficulty practice problems
- Common interview traps and how to dodge them

Show up ready to write SQL on a whiteboard. Tomorrow's capstone uses everything.
— Siraj Sir
#Day28 #PostgreSQL #InterviewPrep

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap — earlier sessions	10 mins	Topic map across all 27 days, Skills checklist before interviews
Part 1: Rapid-Fire Interview Problems	60 mins	9 problems, one per major topic, Top-N, MoM growth, cohort, dedup, EXISTS, median, pivot, recursive, EXPLAIN, Each ~5-7 min to solve
Part 2: Explain-Your-Query Drill	30 mins	Pick any query from the course, Walk through line-by-line as if explaining to interviewer, Naming, order of operations, edge cases, alternatives
Part 3: Mixed Practice (6 problems)	30 mins	Multi-concept problems combining everything, Real interview-style scenarios
Wrap + Capstone Preview	10 mins	Common traps recap, the capstone capstone overview

YOUR SQL JOURNEY

- ✓ Median + DISTINCT ON
- ✓ Date/Time Mastery
- ✓ JSON
- ✓ Views & MVs
- ✓ Data Quality
- ✓ Cohort, RFM, Funnels
- 28 **Interview Prep** ← HERE
- 29 Capstone SQL Intro

RECAP

Memory Check

earlier (earlier sessions): SQL intro → setup → DDL → constraints → filtering. The basics.

earlier (earlier sessions): Scalar functions → CASE → aggregates → JOINs Part 1 → JOINs Part 2. Querying multiple tables.

earlier (earlier sessions): SELF JOIN/UNION → Subqueries Part 1 → Subqueries Part 2/CTEs → DB Concepts → Review. Composability.

earlier (earlier sessions): Window functions Part 1/2/3 → Query plans → Indexing. Performance + analytics.

earlier (earlier sessions): Pivot/FILTER → Median/DISTINCT ON → Date/Time → JSON → Views/MVs. Analyst power tools.

earlier (earlier sessions): Data Quality → Cohort/RFM/Funnels → Interview Prep → Capstone.

You now have every tool a working data analyst uses daily.

STORY

What Interviewers Actually Test

Junior analyst interviews focus on: 'can you write a working SELECT? Do you understand JOINs?' Senior analyst interviews focus on: 'can you EXPLAIN why this query is wrong? Can you spot the bug? Can you propose three different solutions and pick the best one?'

Today's drill is designed for the SENIOR level. Each problem has more than one correct answer. The interviewer is watching how you THINK, not just what you write. You'll narrate the trade-offs, the edge cases, and the alternatives.

Pro tip: always say 'Let me think about edge cases' BEFORE writing the query. NULLs, empty groups, ties, division by zero, fan-out, type mismatches. Interviewers love candidates who name traps before stepping into them.

REVEAL

The 9 Pattern Tests

1. Top-N per group → ROW_NUMBER + PARTITION BY
2. Month-over-month growth → LAG OVER ORDER BY date
3. Cohort retention → 3-CTE pattern
4. Dedup keep latest → ROW_NUMBER + filter rn=1
5. EXISTS vs IN → choose by NULL safety + size
6. Median per group → PERCENTILE_CONT WITHIN GROUP
7. Pivot with FILTER → conditional aggregation
8. Recursive CTE → hierarchy or series traversal
9. EXPLAIN plan reading → bottleneck identification

Problem 1: Top 3 Highest-Paid Employees Per Department

MEDIUM

SCENARIO: HR wants the top 3 paid employees in EACH department — handling ties naturally with DENSE_RANK.

TASK: Use a window function inside a CTE. Filter outer query to rank ≤ 3.

HINT: Top-N-per-group ALWAYS = ranking window function inside a CTE, then WHERE rank <= N. Trap: ROW_NUMBER breaks ties arbitrarily; RANK leaves gaps; DENSE_RANK keeps tied positions together without gaps — pick based on whether ties matter.

Answer

✅ ANSWER

```
WITH ranked AS (
  SELECT
    d.dept_name,
    e.first_name || ' ' || e.last_name
      AS employee,
    e.salary,
    DENSE_RANK() OVER (
      PARTITION BY e.dept_id
      ORDER BY e.salary DESC) AS rk
  FROM stores.employees e
  JOIN core.dim_department d
    ON e.dept_id = d.dept_id
)
SELECT dept_name, employee, salary, rk
FROM ranked
WHERE rk <= 3
ORDER BY dept_name, rk;
```

Problem 2: Month-over-Month Revenue Growth %

MEDIUM

SCENARIO: The CFO wants monthly revenue with MoM growth %. Classic LAG pattern.

TASK: Aggregate revenue per month. Use LAG to compare to previous month. Compute growth %.

HINT: LAG with ORDER BY month inside the window. Trap: integer division returns 0 for any growth under 100% — multiply by 100.0 (not 100) to force float arithmetic before dividing.

Answer

✅ ANSWER

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date)::DATE
      AS month,
    SUM(net_total) AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
  month,
  ROUND(revenue, 0) AS revenue,
  ROUND(LAG(revenue) OVER (
    ORDER BY month), 0) AS prev_month,
  ROUND(100.0 * (revenue
    - LAG(revenue) OVER (ORDER BY month))
    / NULLIF(LAG(revenue) OVER (
      ORDER BY month), 0), 1)
    AS mom_growth_pct
FROM monthly
ORDER BY month;
```

Problem 3: Cohort Retention — Month 0 vs Month 1

HARD

SCENARIO: Quick cohort sanity check: for customers acquired in 2024, what % returned in the month AFTER their first order?

TASK: Compute cohort_month per customer. Count distinct customers active in month 0 and month 1. Show retention %.

HINT: Two CTEs — define cohorts, measure activity. Trap: month 0 should always be ~100% of the cohort (it's their first order BY DEFINITION); if it isn't, your cohort_month definition is wrong.

Answer (1/2)

✅ ANSWER

```
WITH cohorts AS (
  SELECT cust_id,
    DATE_TRUNC('month',
      MIN(order_date))::DATE AS cohort_month
  FROM sales.orders
  WHERE EXTRACT(YEAR FROM order_date) = 2024
  GROUP BY cust_id
),
activity AS (
  SELECT c.cohort_month,
    c.cust_id,
    EXTRACT(MONTH FROM AGE(
      o.order_date, c.cohort_month))
      AS months_since
  FROM cohorts c
  JOIN sales.orders o
    ON c.cust_id = o.cust_id
)
SELECT
  cohort_month,
  COUNT(DISTINCT CASE WHEN months_since = 0
    THEN cust_id END) AS m0,
```

Answer (2/2)

✅ ANSWER

```
  COUNT(DISTINCT CASE WHEN months_since = 1
    THEN cust_id END) AS m1,
  ROUND(100.0 * COUNT(DISTINCT
    CASE WHEN months_since = 1
    THEN cust_id END)
    / NULLIF(COUNT(DISTINCT
    CASE WHEN months_since = 0
    THEN cust_id END), 0), 1)
    AS m1_retention_pct
FROM activity
GROUP BY cohort_month
ORDER BY cohort_month;
```

Problem 4: Deduplicate Customers Keep Latest Per Email

MEDIUM

SCENARIO: customers table has duplicate emails (data error). Keep only the most recently registered record per email.

TASK: ROW_NUMBER over (PARTITION BY email ORDER BY registration_date DESC). Keep rn = 1.

HINT: Dedup-keep-latest is THE textbook ROW_NUMBER pattern. Trap: NULL emails will partition together — every NULL-email row becomes 'duplicate'; filter WHERE email IS NOT NULL OR handle them in their own bucket explicitly.

Answer

✅ ANSWER

```
WITH ranked AS (
  SELECT
    customer_id, email, registration_date,
    ROW_NUMBER() OVER (
      PARTITION BY LOWER(TRIM(email))
      ORDER BY registration_date DESC,
        customer_id DESC) AS rn
  FROM customers.customers
  WHERE email IS NOT NULL
)
SELECT customer_id, email, registration_date
FROM ranked
WHERE rn = 1
ORDER BY registration_date DESC LIMIT 15;
```

Problem 5: EXISTS vs IN Rewrite

MEDIUM

SCENARIO: 'Find customers who placed an order.' Show TWO equivalent queries — IN-subquery and EXISTS — then explain which to prefer.

TASK: Write both versions. State performance/NULL trade-offs.

HINT: Same business question, two SQL shapes. Trap: NOT IN with a NULL inner value silently returns ZERO rows; NOT EXISTS is NULL-safe and usually faster — the NULL-safety alone makes EXISTS the safer default for exclusion queries.

Answer

✅ ANSWER

```
-- Version A: IN-subquery
SELECT c.customer_id, c.first_name
FROM customers.customers c
WHERE c.customer_id IN (
  SELECT cust_id FROM sales.orders);

-- Version B: EXISTS (correlated)
SELECT c.customer_id, c.first_name
FROM customers.customers c
WHERE EXISTS (
  SELECT 1 FROM sales.orders o
  WHERE o.cust_id = c.customer_id);

-- Both return the same rows.
-- EXISTS is often faster (stops at first
-- match) and NOT EXISTS handles NULLs
-- correctly while NOT IN does not.
```

Problem 6: Median + P95 Order Value Per Region

HARD

SCENARIO: The CFO wants median order value (more robust than average) and P95 (the high-spender benchmark) per region.

TASK: PERCENTILE_CONT(0.5) and PERCENTILE_CONT(0.95) WITHIN GROUP. Group by region_name.

HINT: PERCENTILE_CONT is an ordered-set aggregate — needs the WITHIN GROUP (ORDER BY x) syntax, NOT a regular OVER. Trap: forgetting the ORDER BY inside WITHIN GROUP throws a syntax error — the ORDER BY tells the function what to sort by before picking the percentile point.

Answer

✅ ANSWER

```
SELECT
  r.region_name,
  ROUND((PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY o.net_total))::NUMERIC, 0)
    AS median_order,
  ROUND((PERCENTILE_CONT(0.95)
    WITHIN GROUP (ORDER BY o.net_total))::NUMERIC, 0)
    AS p95_order,
  COUNT(*) AS order_count
FROM sales.orders o
JOIN stores.stores s
  ON o.store_id = s.store_id
JOIN core.dim_region r
  ON s.region_id = r.region_id
GROUP BY r.region_name
ORDER BY median_order DESC;
```

Problem 7: Pivot Order Counts by Status (FILTER Clause)

MEDIUM

SCENARIO: Build a per-store dashboard showing one column per order status: Delivered count, Pending count, Cancelled count, Returned count.

TASK: Use FILTER (WHERE status = ...) inside COUNT(). Group by store.

HINT: FILTER is the modern, clean syntax for conditional aggregation — alternative to CASE-inside-COUNT. Trap: spelling/casing of status values matters — 'Delivered' vs 'delivered' are different; use UPPER or LOWER to normalize if the source data is inconsistent.

Answer

✅ ANSWER

```
SELECT
  s.store_name,
  COUNT(*) FILTER (
    WHERE o.order_status = 'Delivered')
    AS delivered,
  COUNT(*) FILTER (
    WHERE o.order_status = 'Pending')
    AS pending,
  COUNT(*) FILTER (
    WHERE o.order_status = 'Cancelled')
    AS cancelled,
  COUNT(*) FILTER (
    WHERE o.order_status = 'Returned')
    AS returned,
  COUNT(*) AS total
FROM sales.orders o
JOIN stores.stores s
  ON o.store_id = s.store_id
GROUP BY s.store_name
ORDER BY total DESC LIMIT 15;
```

Problem 8: Recursive CTE — Generate Last 12 Months

HARD

SCENARIO: Generate a list of the last 12 months as a date series (for filling in gaps in a sparse monthly report).

TASK: Recursive CTE: anchor = current month, recursive step = previous month, terminate at 12 months back.

HINT: Recursive CTE = anchor + UNION ALL + recursive step + termination. Trap: forgetting the termination condition (WHERE n < 12 or similar) causes infinite recursion; PostgreSQL eventually errors out but the query hangs first.

Answer

✅ ANSWER

```
WITH RECURSIVE months AS (
  SELECT
    DATE_TRUNC('month',
      CURRENT_DATE)::DATE AS month,
    0 AS offset_n
  UNION ALL
  SELECT
    (month - INTERVAL '1 month')::DATE,
    offset_n + 1
  FROM months
  WHERE offset_n < 11
)
SELECT month,
  TO_CHAR(month, 'Mon YYYY') AS label
FROM months
ORDER BY month;
```

Problem 9: Read an EXPLAIN ANALYZE Plan

CRAZY

SCENARIO: A query is slow. Read the plan, identify the bottleneck, propose a fix.

TASK: Run EXPLAIN ANALYZE on the query below. State: total time, most expensive node, estimated vs actual row count mismatch, and what you would change.

HINT: Read plans bottom-up; the slowest node (highest 'actual time') is usually a Seq Scan on a large table or a Nested Loop where Hash Join would be better. Trap: Seq Scan isn't ALWAYS bad — when most rows match (>20%), Seq Scan beats Index Scan; check selectivity before assuming an index will help.

Answer (1/2)

✅ ANSWER

```
EXPLAIN ANALYZE
SELECT
  cat.category_name,
  COUNT(*),
  SUM(oi.net_amount) AS revenue
FROM sales.order_items oi
JOIN products.products p
  ON oi.prod_id = p.product_id
JOIN core.dim_brand b
  ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
JOIN sales.orders o
  ON oi.order_id = o.order_id
WHERE o.order_status = 'Delivered'
  AND o.order_date >= '2025-01-01'
GROUP BY cat.category_name
ORDER BY revenue DESC;

-- Things to look for in the plan:
-- 1. Largest 'actual time' = bottleneck
-- 2. Seq Scan on sales.orders without
```

Answer (2/2)

✅ ANSWER

```
--    an index on order_status + order_date
--    suggests creating that composite index
-- 3. Big gap between estimated vs actual
--    rows = ANALYZE the table to update stats
```

STORY

The Senior Interviewer's Favorite Test

'Walk me through this query line by line.' That's how senior interviews actually work. You don't just write SQL — you NARRATE it. Why this JOIN type? Why this WHERE filter here vs in HAVING? What happens if a column is NULL? What ALTERNATIVE could you write instead?

Today's drill: pick a query you wrote in homework. Practice narrating it as if explaining to a senior. The script template below works for ANY query.

DEFINITION

The 5-Step Explain Script

Step 1: 'Here's the business question.' One sentence.

Step 2: 'My approach is X' — name the pattern (top-N, anti-join, cohort, etc.)

Step 3: Walk through CTEs/subqueries top-down. For each, say: input, transformation, output.

Step 4: 'Edge cases I considered:' NULLs, empty groups, ties, division by zero, fan-out.

Step 5: 'Alternatives:' name one different SQL shape that achieves the same thing.

Example: Narrating Problem 1 (Top-3 Per Department) (1/2)

```
-- Business: 'Top 3 paid employees per department.'

-- Approach: Top-N-per-group pattern.
-- DENSE_RANK because if multiple employees
-- share a salary I want all of them, not break
-- ties arbitrarily.

-- CTE 'ranked':
--   Input: employees + dim_department.
--   Transform: DENSE_RANK partitioned by dept,
--     ordered by salary descending.
--   Output: every employee labeled 1, 2, 3, ...

-- Final SELECT: keep only rank <= 3.

-- Edge cases:
-- - Tied salaries: DENSE_RANK keeps them together
-- - Empty dept: simply won't appear (INNER JOIN)
-- - Department with < 3 employees: shows all of them
```

Example: Narrating Problem 1 (Top-3 Per Department) (2/2)

```
-- Alternative: a correlated subquery counting
-- 'employees with higher salary in same dept' < 3.
-- Window function is cleaner and usually faster.
```

Mixed 1: Find Customers Who Reviewed Without Ordering

EASY

SCENARIO: Data quality check: find customers who wrote reviews but never placed an order — suspicious activity worth investigating.

YOUR TASK: Use NOT EXISTS for the anti-join. Show name, email, count of reviews.

🤔 **Hint:** 'Reviewed but never ordered' = EXISTS in reviews + NOT EXISTS in orders. Trap: a customer with reviews AND orders (the normal case) shouldn't appear; verify with a quick COUNT sanity check before assuming the query is right.

Answer

✅ ANSWER

```
SELECT c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  c.email,
  (SELECT COUNT(*)
   FROM customers.reviews r
   WHERE r.customer_id = c.customer_id)
    AS review_count
FROM customers.customers c
WHERE EXISTS (
  SELECT 1 FROM customers.reviews r
  WHERE r.customer_id = c.customer_id)
AND NOT EXISTS (
  SELECT 1 FROM sales.orders o
  WHERE o.cust_id = c.customer_id)
LIMIT 15;
```

Mixed 2: 7-Day Moving Average of Orders

MEDIUM

SCENARIO: The Ops team wants to smooth out daily order spikes by computing a 7-day rolling average. Reveals true trend underneath day-of-week noise.

YOUR TASK: Aggregate daily orders. Use AVG OVER with ROWS BETWEEN 6 PRECEDING AND CURRENT ROW.

🤔 **Hint:** Moving average = AVG OVER with a frame. Trap: the default frame for AVG OVER (with ORDER BY) is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — that's a CUMULATIVE average, not a moving one. Explicitly specify ROWS BETWEEN N PRECEDING AND CURRENT ROW.

Answer

✅ ANSWER

```
WITH daily AS (
  SELECT
    order_date::DATE AS day,
    COUNT(*) AS orders
  FROM sales.orders
  WHERE order_date >= CURRENT_DATE - 60
  GROUP BY order_date::DATE
)
SELECT day, orders,
  ROUND(AVG(orders) OVER (
    ORDER BY day
    ROWS BETWEEN 6 PRECEDING
    AND CURRENT ROW), 1) AS ma_7d
FROM daily
ORDER BY day DESC LIMIT 30;
```

Mixed 3: RFM Champions Receiving No Recent Email

HARD

SCENARIO: Marketing wants to find RFM Champions (top R+F+M) who haven't received an email campaign in the last 30 days — a critical re-engagement gap.

YOUR TASK: CTE 1: RFM scoring. CTE 2: champions (R=5, F=5, M=5). Anti-join against marketing.email_clicks for recent campaigns.

🤔 **Hint:** Three patterns in one query: NTILE for RFM, anti-join via NOT EXISTS, date filtering. Trap: marketing.email_clicks is aggregated per campaign per day — it doesn't store per-customer sends; this query assumes campaigns reach all members, so the actual implementation may need a richer email-recipient table in production.

Answer (1/2)

✅ ANSWER

```
WITH rfm AS (
  SELECT cust_id,
    NTILE(5) OVER (
      ORDER BY MAX(order_date)) AS r,
    NTILE(5) OVER (
      ORDER BY COUNT(*)) AS f,
    NTILE(5) OVER (
      ORDER BY SUM(net_total)) AS m
  FROM sales.orders
  GROUP BY cust_id
)
SELECT
  c.first_name || ' ' || c.last_name
    AS champion,
  c.email, rfm.r, rfm.f, rfm.m
FROM rfm
JOIN customers.customers c
  ON rfm.cust_id = c.customer_id
WHERE rfm.r = 5 AND rfm.f = 5
  AND rfm.m = 5
  AND NOT EXISTS (
    SELECT 1 FROM marketing.email_clicks ec
```

Answer (2/2)

✅ ANSWER

```
    WHERE ec.sent_date
      >= CURRENT_DATE - 30
  )
LIMIT 15;
```

Mixed 4: Categories Sold in Both 2024 AND 2025

MEDIUM

SCENARIO: Merchandising wants 'evergreen' categories — sold in BOTH years. Set-comparison classic.

YOUR TASK: Two CTEs (one per year) listing distinct category_ids. INNER JOIN them — categories present in BOTH.

🤔 **Hint:** Set-intersection problem. Could use INTERSECT, JOIN, or EXISTS — each gives the same result. Trap: with INTERSECT, both SELECTs must return identical column shapes; with JOIN, INNER drops categories present in only one year (which is what we want here).

Answer (1/2)

✅ ANSWER

```
WITH cat_2024 AS (
  SELECT DISTINCT cat.category_id,
    cat.category_name
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
  JOIN core.dim_category cat
    ON b.category_id = cat.category_id
  WHERE EXTRACT(YEAR FROM o.order_date) = 2024
),
cat_2025 AS (
  SELECT DISTINCT b.category_id
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  JOIN core.dim_brand b
```

Answer (2/2)

✅ ANSWER

```
    ON p.brand_id = b.brand_id
  WHERE EXTRACT(YEAR FROM o.order_date) = 2025
)
SELECT c24.category_name
FROM cat_2024 c24
JOIN cat_2025 c25
  ON c24.category_id = c25.category_id
ORDER BY c24.category_name;
```

Mixed 5: Stores with Below-Median Per-Order Revenue

HARD

SCENARIO: Ops wants stores whose average order value is below the COMPANY median. The 'underperforming locations' list.

YOUR TASK: CTE 1: per-store AVG net_total. Scalar subquery: company-wide median AVG. Filter stores with per-store AVG < median.

🤔 **Hint:** 'Median of averages' is a two-step calculation — first per-store AVG, then median of those AVGs. Trap: median of all orders' net_total (one-step) gives a DIFFERENT number than median of per-store AVGs — the question is about store performance, so it's the second.

Answer

✅ ANSWER

```
WITH store_avg AS (
  SELECT store_id,
    AVG(net_total) AS avg_order
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY store_id
)
SELECT
  s.store_name, s.city,
  ROUND(sa.avg_order, 0) AS avg_order,
  (SELECT ROUND((PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY avg_order))::NUMERIC, 0)
   FROM store_avg)
    AS company_median
FROM store_avg sa
JOIN stores.stores s
  ON sa.store_id = s.store_id
WHERE sa.avg_order < (
  SELECT PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY avg_order)
  FROM store_avg)
ORDER BY sa.avg_order LIMIT 15;
```

Mixed 6: Build a Multi-Source Customer 360

CRAZY

SCENARIO: Senior interview classic: build a customer-360 row showing name, total spend, last order date, support ticket count, review count, loyalty tier, and stickiness score. ALL data sources, one row per customer.

YOUR TASK: Pre-aggregate each source in its own CTE. LEFT JOIN all CTEs to customers. COALESCE missing values to 0. Show top 15 by total spend.

🤔 **Hint:** Multi-source dashboards must use PRE-AGGREGATION CTEs to avoid fan-out — joining raw orders + tickets + reviews directly multiplies rows. Trap: COALESCE(..., 0) on COUNTs and SUMs prevents NULL columns for customers missing from a source; without it, the report has gaps that look like data quality issues.

Answer (1/2)

✅ ANSWER

```
WITH spend AS (
  SELECT cust_id,
    SUM(net_total) AS total_spend,
    MAX(order_date) AS last_order
  FROM sales.orders
  GROUP BY cust_id),
tickets AS (
  SELECT customer_id, COUNT(*) AS ticket_count
  FROM support.tickets
  GROUP BY customer_id),
reviews AS (
  SELECT customer_id, COUNT(*) AS review_count
  FROM customers.reviews
  GROUP BY customer_id)
SELECT
  c.first_name || ' ' || c.last_name
    AS customer,
  ROUND(COALESCE(s.total_spend, 0), 0)
    AS spend,
  s.last_order,
  COALESCE(t.ticket_count, 0) AS tickets,
  COALESCE(r.review_count, 0) AS reviews,
```

Answer (2/2)

✅ ANSWER

```
  COALESCE(lt.tier_name, 'Non-Member')
    AS tier
FROM customers.customers c
LEFT JOIN spend s
  ON c.customer_id = s.cust_id
LEFT JOIN tickets t
  ON c.customer_id = t.customer_id
LEFT JOIN reviews r
  ON c.customer_id = r.customer_id
LEFT JOIN loyalty.members m
  ON c.customer_id = m.customer_id
LEFT JOIN loyalty.tiers lt
  ON m.tier_id = lt.tier_id
ORDER BY spend DESC LIMIT 15;
```

COMMON MISTAKE

Trap 1: NULL in NOT IN

WHERE id NOT IN (SELECT col FROM t) — if ANY value in the inner list is NULL, the whole filter returns UNKNOWN → FALSE for every outer row. Result: zero rows.

Fix: WHERE id IS NOT NULL in the inner query, OR switch to NOT EXISTS (NULL-safe).

COMMON MISTAKE

Trap 2: WHERE on Right Table in LEFT JOIN

LEFT JOIN orders ON … WHERE o.status = 'Delivered' silently converts to INNER JOIN — unmatched left rows have NULL status, which fails the equality check.

Fix: Move the filter into the ON clause, OR use COALESCE in WHERE.

COMMON MISTAKE

Trap 3: Integer Division

ROUND(100 * count / total, 1) — if count and total are integers, the division truncates to 0 for anything under 100%.

Fix: Multiply by 100.0 (not 100). Or cast one side: count::NUMERIC / total.

COMMON MISTAKE

Trap 4: Fan-Out in Multi-Source Aggregates

JOIN orders, items, payments all to customers → row count multiplies. SUM(net_total) gets inflated.

Fix: Pre-aggregate each source in its own CTE, then JOIN the summaries.

COMMON MISTAKE

Trap 5: NULL vs Empty String

WHERE email = '' will MISS rows where email IS NULL — they're different things in SQL.

Fix: Use COALESCE(email, '') = '' OR explicitly handle WHERE email IS NULL OR email = ''.

COMMON MISTAKE

Trap 6: Aggregate in WHERE

WHERE COUNT(*) > 10 — throws an error. WHERE runs BEFORE aggregation.

Fix: Use HAVING for aggregate filters: HAVING COUNT(*) > 10 after GROUP BY.

REAL WORLD

What You Achieved Across 28 Days

earlier — Setup and foundations. earlier — Querying single and multiple tables. earlier — Composition (subqueries, CTEs, set ops). earlier — Window functions + performance. earlier — Analyst power tools (pivot, dates, JSON, views). earlier — Data quality, cohorts, RFM, funnels, interview prep.

You have walked through ~150 hands-on problems, fixed bugs in broken queries, debugged EXPLAIN plans, and applied every major SQL pattern an analyst uses. The shape of any interview question is now familiar to you.

Tomorrow (the capstone): Capstone SQL Intro — build the analytics layer. Views, materialized views, JSON exports. the capstone: Dashboard + deploy + demo. Your portfolio piece for every job interview.

KEY TAKEAWAYS

INTERVIEW SHOWUP CHECKLIST

Before walking into ANY SQL interview:

1. Practice the 9 problems from Part 1 on a whiteboard until you can write them from memory.
2. Practice the 'Explain Your Query' drill — narrate at least 3 of your homework queries out loud.
3. Know the 6 traps cold — be the candidate who NAMES them before stepping into them.
4. Have ONE answer ready for 'tell me about a SQL problem you solved' — pick a homework problem you remember well.
5. Sleep well. Drink water. Smile. You know this material.

STORY

Coming Tomorrow

Capstone SQL Intro: build the analytics_schema. Create 15+ views and 6+ materialized views covering Sales, Customer, Product, Store, Finance, Supply Chain, Audit modules. Refresh script. JSON export.

Dashboard build + GitHub Pages deploy + demo. Your portfolio piece for every job interview.